

Übungsblatt 6

Ü 6.1 Theoretische Fragen zu Multi-Prozessor-Programmierung

- In den letzten 10 Jahren ist die Anzahl der Prozessor Kerne für x86-Architekturen angestiegen, vor 10 Jahren von 4-8 Kernen sehen wir nun vermehrt ≥ 32 Kerne, aber warum sehen wir keine 256 Kern CPUs? Welche Herausforderungen an die Softwareentwicklung würden sich stellen?
- Eine Domäne, in der wir sehr viele Prozessor-Kerne sehen, sind GPUs; hier werden Chips und Architekturen für spezielle Aufgaben verwendet (Siehe Nvidia H100). Welche Herausforderungen würden entstehen, wenn wir uns in der CPU-Architektur diesem Paradigma anschließen würden?

Ü 6.2 Prozesse teilen sich eine Datei

Erstellen Sie ein C-Programm `abwechselnd.c` welches über zwei Prozesse verfügt. Folgende Tätigkeit soll von den Prozessen ausgeführt werden:

- Der Hauptprozess erstellt eine Datei mit dem Namen `data.txt` und befüllt diese mit einer beliebigen Zeichenfolge (zumindest 20 Zeichen, Daten selbst wählen).
- Der Kindprozess soll dieselbe Datei lesen, den Inhalt invertieren (aus 'ABC' wird also 'CBA') und den Dateiinhalt überschreiben.
- Der Hauptprozess soll danach die invertierten Daten herauslesen, ausgeben und aus der Datei löschen. Der gesamte Prozess soll mindestens fünfmal wiederholt werden.
- Stellen Sie mit IPC sicher, dass beide Prozesse immer abwechselnd mit der Datei interagieren.

Ü 6.3 Datei durch Shared Memory ersetzen

Shared Memory ist auch eine häufige Methode, um Daten zwischen Prozessen auszutauschen. Verwenden Sie Ihre Lösung aus **6.2 (Processes teilen sich eine Datei)**, um Folgendes in einem Programm `abwechselnd_shared.c` zu erreichen:

- Ersetzen Sie die Verwendung der Datei durch Shared Memory (Inspiration finden Sie hier: <https://users.cs.cf.ac.uk/Dave.Marshall/C/node27.html>). Achten Sie nach Ablauf Ihres Programms auf Trennung und Bereinigung.
- Vergleichen Sie die Performance beider Implementierungen! Welche ist performanter? (z. B. mit dem Befehl `time ./abwechselnd_shared`)
- Verändert sich nur die Performance oder gibt es auch weitere Vorteile, die durch die Verwendung von Shared Memory entstehen?

Ü 6.4 Named Pipes

Ein weiteres Paradigma der Interprozesskommunikation (IPC) sind Named Pipes (FIFO-Dateien). Im Gegensatz zu anonymen Pipes bleiben Named Pipes im Dateisystem bestehen und ermöglichen die Kommunikation zwischen Prozessen, auch wenn diese nicht direkt miteinander verwandt sind. Ein häufig verwendetes Modell in diesem Zusammenhang ist das Client-Server-Prinzip:

- Der **Server** erstellt eine Named Pipe und wartet auf eingehende Nachrichten.
- Der **Client** öffnet diese Named Pipe und sendet Daten.
- Eine **Named Pipe** kann immer nur von einem Prozess zum Schreiben und von einem anderen zum Lesen geöffnet werden.

Implementieren Sie ein C-Programm, das mithilfe von Named Pipes Zeichenketten zwischen einem Client- und einem Server-Prozess austauscht. Der Name der Pipe kann statisch vergeben werden, z.B. `/tmp/myfifo`. Die vom Client gesendete Zeichenkette darf frei gewählt oder eingegeben werden.

Führen Sie folgende Tests durch und analysieren Sie das Verhalten:

1. Zeigen Sie, dass Ihr Programm Daten an sich selbst senden kann (z.B. Server und Client innerhalb derselben Anwendung).
2. Starten Sie eine zweite Instanz Ihres Programms und lassen Sie diese Daten an den ersten Serverprozess senden. Welche Probleme treten dabei auf?
3. Überlegen Sie, welcher Nachteil entstehen würde, wenn Sie Server und Client in getrennte Programme auslagern würden.

Zusätzliche Informationen:

- Sie können die Implementierung sowohl mit `mkfifo()` (d.h., die Named Pipe wird innerhalb des Programms erstellt) als auch mit einer extern erstellten Pipe via Terminal (`mkfifo /tmp/myfifo`) angehen.
- Dokumentieren Sie Ihre Arbeitsschritte und Ergebnisse in geeigneter Weise (z.B. durch Screenshots).
- Hinweis: Eine Named Pipe kann oft nicht innerhalb eines Shared Folders einer VM angelegt werden, sondern nur innerhalb des nativen Dateisystems.

Ü 6.5 pipe + dup2

Machen Sie sich mit der Funktion `dup2` vertraut.

Schreiben Sie ein C-Programm `myduppipe.c` (basierend auf der Vorlage `myduppipe_stub.c`), das zwei Prozesse erstellt, welche Systemprogramme ausführen und die über eine **unidirektionale Pipe** kommunizieren.

Das Programm erhält beim Start zwei **Programmnamen** A und B (inkl. ihrer Parameter) als **Kommandozeilenargumente**. Der **Elternprozess** erzeugt dann zwei **Kindprozesse, welche die Programme ausführen**:

- Der erste Kindprozess führt **Programm A** aus und sendet die Ausgabe an den zweiten Kindprozess mittels der unidirektionalen Pipe.
- Der zweite Kindprozess erhält die Ausgabe von **A** und führt **Programm B** darauf aus.

Dazu muss die **Standardausgabe von A auf die Standardeingabe von B umgeleitet** werden

Beispiel: Der Aufruf:

```
./myduppipe "ls -al" "sort -r"
```

soll äquivalent zu folgendem Shell-Befehl sein:

```
ls -al | sort -r
```

1. **Implementieren Sie das beschriebene Verhalten in C.**
2. **Skizzieren Sie den Aufbau und die Kommunikationsflüsse** zwischen den Eltern- und Kindprozessen.
3. **Optional:** Erweitern Sie das Programm, um eine beliebige Anzahl von Kommandos zu verketteten.

Beispiel mit mehreren Kommandos:

```
./myduppipe "ps -eo user=" "sort" "uniq -c" "sort -nr"
```

Bonuspunkt: Präsentieren Sie diese Erweiterung in der Lösungsvorstellung.